

Python Lists

A clean, detailed teaching slide deck with examples, practice tasks, and MCQs.

- A list is used to store multiple values in one variable.
- Lists are ordered, changeable, and allow duplicate values.
- Lists can store numbers, strings, booleans, and mixed data.
- We use lists when one variable needs to hold many related items.

① What is a List?

- A list is a collection data types in python.
- It stores many items inside square brackets `[]`.
- Each item has a position number called an index.
- The first item is at index 0, not index 1.

Example

```
fruits = ["apple", "banana", "mango"]
```

index positions

apple → 0

banana → 1

mango → 2

Index	0	1	2
Value	apple	banana	mango

Main Properties of Lists

Property	Meaning	Example
Ordered	Items keep their position	[0, 20, 30]
Changeable	We can update items	marks[0] = 95
Allows duplicates	Same value can appear many times	[5, 5, 10]
Mixed types	Different data types can be stored	[1, "Ram", True]
Indexed	Items are accessed using index	names[1]

```
numbers = [10, 20, 20, 30] # ordered and duplicate values allowed
```

Creating Lists

- Use Square brackets `[]` to create a list.
- Separate items using commas.
- An empty list can be created first and filled later.
- `list()` can also create a list from some iterable values.

Code examples

```
# list of integers
```

```
marks = [70, 85, 90]
```

```
# list of strings
```

```
subjects = ["Python", "math", "AI"]
```

```
# empty list  
items = []
```

```
# mixed list
```

```
student = ["Asha", 21, 3.8, True]
```

```
print(type(subjects))
```

Accessing Items Using Index

- Index is the position of an item in a list.
- positive indexing start from 0.
- negative indexing start from -1 for the last item.
- use list name [index] to access a single item.

Code example

```
color = ["red", "green", "blue"]
print(colors[0]) # red
print(colors[1]) # green
print(colors[-1]) # blue
print(colors[-2]) # green
```

Value	red	green	blue	
Index	0	1	2	
Negative	-3	-2	-1	

List Slicing

- Slicing means taking a part of a list.
- Syntax: list[start:stop:step]
- Start is included, stop is excluded.
- If start is missing, Python starts from the beginning.
- If stop is missing, Python goes to the end.

Code example

```
num = [10, 20, 30, 40, 50]
print(nums[1:4]) # [20, 30, 40]
print(nums[:3]) # [10, 20, 30]
print(nums[2:]) # [30, 40, 50]
print(nums[:2]) # [10, 30, 50]
print(nums[::-1]) # [reverse list]
```

Remember: Stop index is not included

* Updating List Items

- List are mutable, so we can change their values.
- Use index to update one item `marks[1] = 95`
- Use Slicing to update many items at once.
- The original list changes after update.

Code example -

```
marks = [60, 70, 80]
```

```
marks[1] = 95
```

```
print(marks)
```

```
# [60, 95, 80]
```

```
marks[0:2] = [50, 55]
```

```
print(marks)
```

```
# [50, 55, 80]
```

Good use: Correcting student marks, updating product price, changing status.

* Adding items to a List

Method	Use	Example
<code>append()</code>	add one item at the end	<code>names.append("Hari")</code>
<code>insert()</code>	add one item at specific index	<code>names.insert(1, "Sita")</code>
<code>extend()</code>	add many items from another list	<code>names.extend(["Gita", "Nita"])</code>

Code example

```
name = ["Ram", "Asha"]
```

```
name.append("Hari")
```

```
name.insert(1, "Sita")
```

```
name.extend(["Gita", "Nita"])
```

```
print(name)
```

Output

```
["Ram", "Sita", "Asha", "Hari", "Gita", "Nita"]
```

* Removing items from a list

method	use	Example
<code>remove(value)</code>	removes first matching value	<code>nums.remove(20)</code>
<code>pop(index)</code>	removes item by index and returns it	<code>nums.pop(1)</code>
<code>pop()</code>	removes the last item	<code>nums.pop()</code>
<code>del</code>	deletes item or whole list	<code>del nums[0]</code>
<code>clear()</code>	removes all items but keep list	<code>nums.clear()</code>

Code example

```
num = [10, 20, 30, 20]
```

```
num.remove(20)
```

```
last = num.pop()
```

```
print(nums)
```

```
print(last)
```

Important

`remove(20)` removes the first 20 only.

`pop()` removes and returns a value.

Important List Methods

Method	meaning	Example
<code>append(x)</code>	add item at end	<code>a.append(5)</code>
<code>insert(i, x)</code>	add item at index i	<code>a.insert(0, 5)</code>
<code>extend(b)</code>	add all items from b	<code>a.extend(b)</code>
<code>remove(x)</code>	remove first x	<code>a.remove(5)</code>
<code>pop(i)</code>	remove by index	<code>a.pop(0)</code>
<code>clear()</code>	empty the list	<code>a.clear()</code>
<code>index(x)</code>	find index of x	<code>a.index(5)</code>
<code>count(x)</code>	count x	<code>a.count(5)</code>
<code>sort()</code>	sort list	<code>a.sort()</code>
<code>reverse()</code>	reverse list	<code>a.reverse()</code>

Teaching point: methods usually change the original list.

Looping Through Lists

- A loop helps us process each item in a list.
- use `for item in list` to read values one by one.
- use `range(len(list))` when the index is also needed.

Loop through values

```
Subjects = ["Python", "DBMS", "AI"]
```

```
for Subject in Subjects:
    print(Subject)
```

Loop with index

```
marks = [75, 80, 90]
```

```
for i in range(len(marks)):
    print(i, marks[i])
```

Output

0 75

1 80

2 90

Sorting and Reversing Lists

- `Sort()` changes the original list.
- `Sorted()` creates a new sorted list.
- `reverse()` reverses the original list order.
- Use `reverse = True` for descending order.

Code example

```
marks = [80, 60, 95, 70]
```

```
marks.sort()
```

```
print(marks)
```

```
# [60, 70, 80, 95]
```

```
marks.sort(reverse=True)
```

```
print(marks)
```

```
# [95, 80, 70, 60]
```

```
new_list = Sorted(marks)
```

Use Sorting for rankings, reports, top marks and ordered output.

Nested Lists

- A nested list is a list inside another list.
- It is useful for table-like data.
- Use two indexes: outer index and inner index
- Examples: Students [0] [1] means row 0, column 1.

Row	Name	Marks
0	Ram	80
1	Sita	90
2	Harish	75

Code example

```
Student = [
    ["Ram", 80],
    ["Sita", 90],
    ["Harish", 75]
```

```
]
```

```
print (Student [0])      # ["Ram", 80]
print (Student [0] [0])  # Ram
print (Student [1] [1])  # 90
```

Copying Lists

- Writing `List 2 = List 1` does not create a new independent copy
- Both names can point to the same list.
- use `Copy()`, `list()`, or slicing `[:]` to create a separate copy.

Same Reference

```
a = [10, 20, 30]
```

```
b = a
```

```
b[0] = 99
```

```
print(a) # [99, 20, 30]
```

Correct Copy

```
a = [10, 20, 30]
```

```
b = a.copy()
```

```
b[0] = 99
```

```
print(a) # [10, 20, 30]
```

```
print(b) # [99, 20, 30]
```